

Coding the Solution to Ireland's Scam Call Issue

Project reference number: PR2878

Stand Number: 2401

Appendices:

Contents:

Page:	Section:
1-7	Product testing survey
8-14	App improvements

Product Testing Survey:

Overview:

After demonstrating of the app to 5 science classes across our school, we carried out a survey which aimed to get an idea of how useful students found the app to be and if they themselves would make use of it.



Distribution:

The survey was sent to students on their science class team on Microsoft Teams after they witnessed the app in action.

Survey questions:

1: Did you find you were aware of the potential threat the warning during a call highlighted?

☐ Yes

☐ No

2: If you answered 'no', how do you think it could be made more obvious?

Enter your answer

3. On a scale of 1-5 how easy (with 5 being very easy and 1 being very hard) was it to understand the urgency of this warning?

☐ 1

☐ 2

☐ 3

☐ 4

☐ 5

4. Do you think you would find this warning to be annoying or useful?

☐ Useful

☐ Annoying

☐ Unsure

5. Did you find that the message asking whether you trust a number was clear enough for you to see and consider responding to?

☐ Yes

☐ No

6: If 'no', how could it become clearer?

Enter your answer

7: How likely would it be (with 5 being very likely and 1 being very unlikely) for you to respond to this question?

- ☐ 1
- ☐ 2
- ☐ 3
- ☐ 4
- ☐ 5

8: On a scale of 1-5, how likely (with 5 being very likely and 1 being very unlikely) would it be for you to download this app on your own phone?

- ☐ 1
- ☐ 2
- ☐ 3
- ☐ 4
- ☐ 5

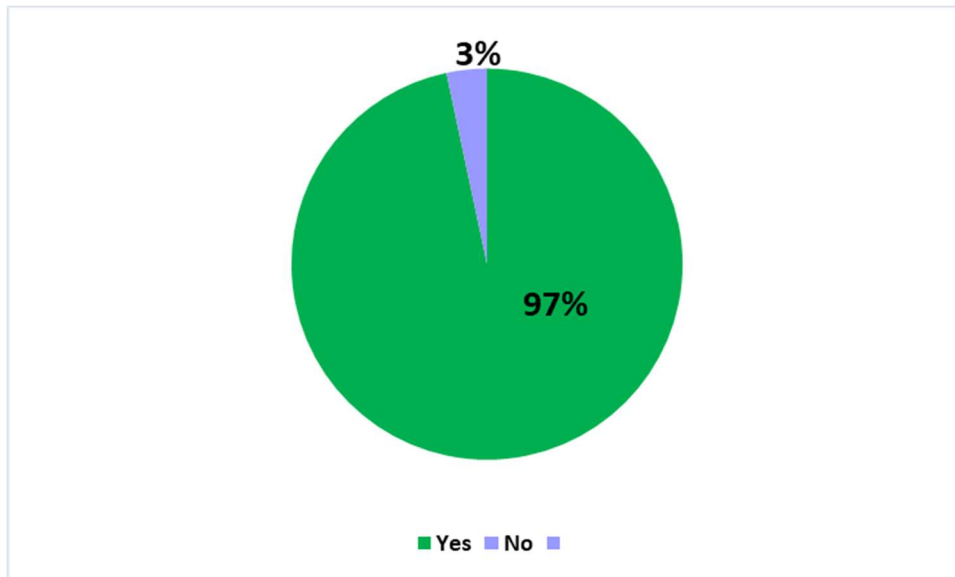
9: What features would you like to see in the in-app UI in future?

- ☐ Call log showing suspicious numbers that have called you and option to block them
- ☐ Links to useful websites containing advice on how to protect yourself against scams
- ☐ Hotline numbers that will help you if you fall victim to a scam.
- ☐ More specific info in warnings based on what kind of scam the number relates to
- ☐ Other

10. Have you any other comments or suggestions in relation to the app?

Survey results:

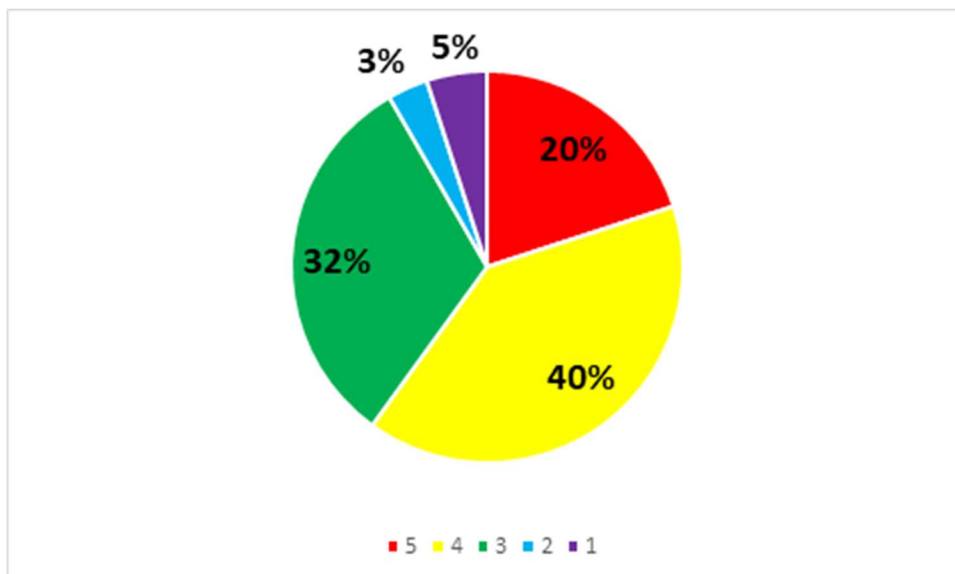
1: Did you find you were aware of the potential threat the warning during a call highlighted?



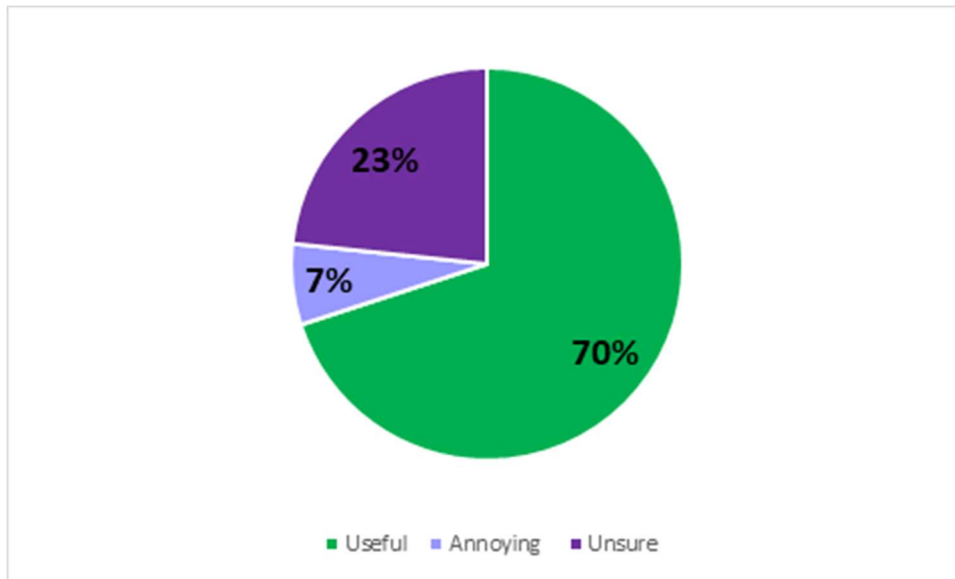
2: If you answered 'no', how do you think it could be made more obvious?

- Highlight it in red.
- Cover the whole screen.
- Make the information more obvious.

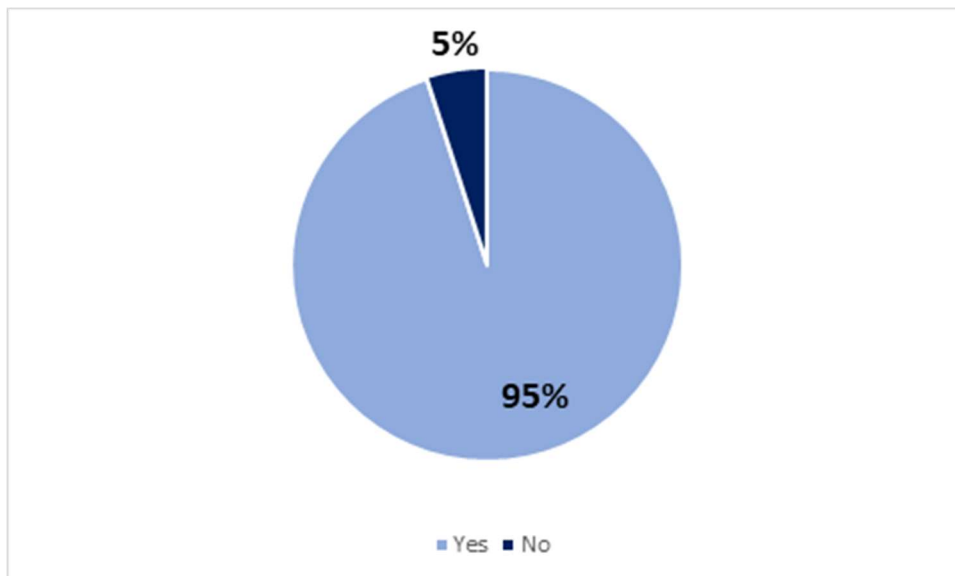
3. On a scale of 1-5 how easy (with 5 being very easy and 1 being very hard) was it to understand the urgency of this warning?



4. Do you think you would find this warning to be annoying or useful?



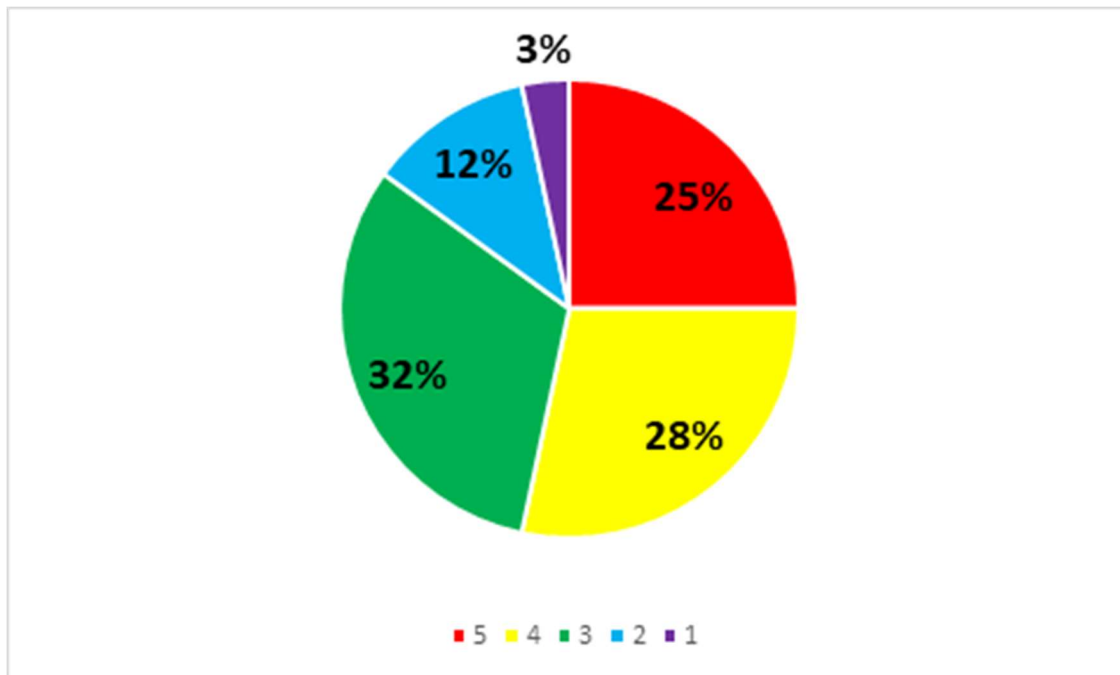
5. Did you find that the message asking whether you trust a number was clear enough for you to see and consider responding to?



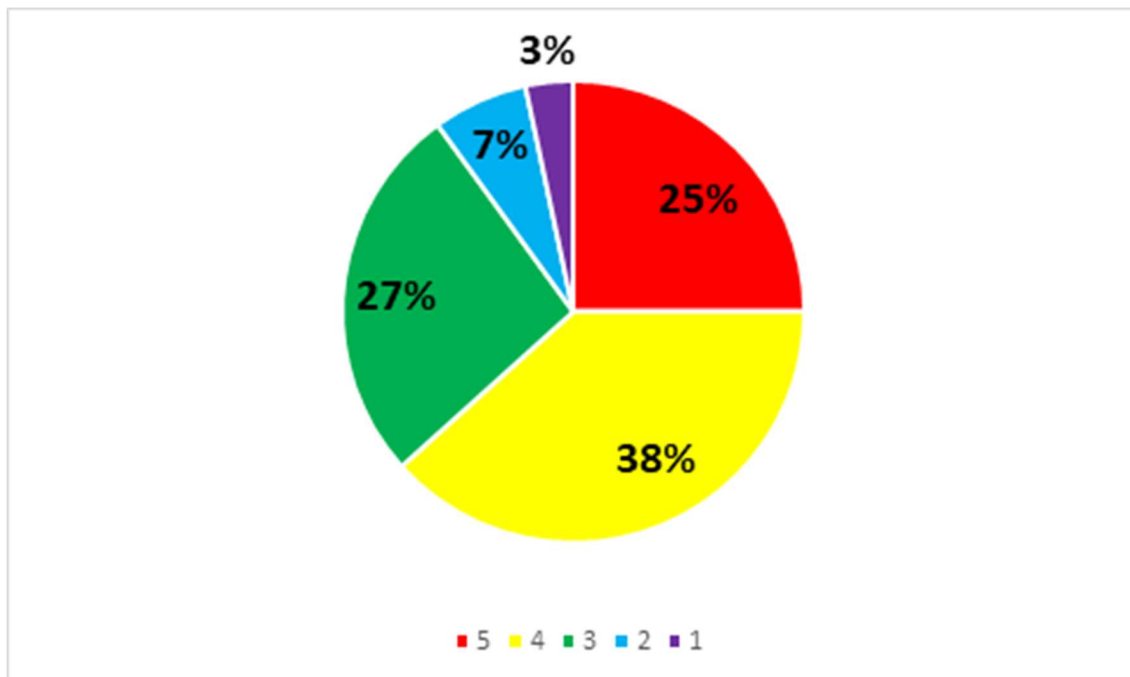
6: If 'no', how could it become clearer?

- It should pop up in the middle so it wouldn't be mistaken for a regular notification.
- It could cover the whole screen.

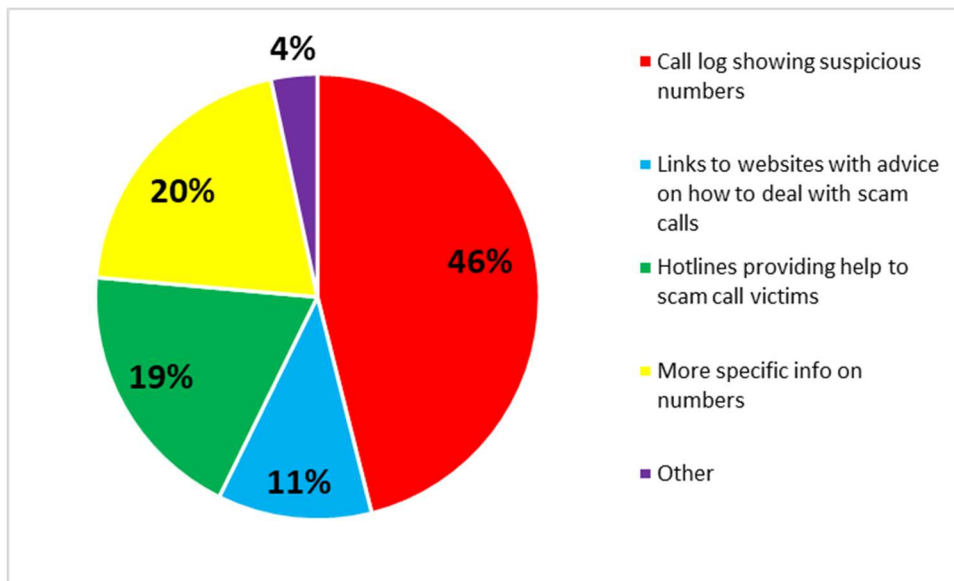
7: How likely would it be (with 5 being very likely and 1 being very unlikely) for you to respond to this question?



8: On a scale of 1-5, how likely (with 5 being very likely and 1 being very unlikely) would it be for you to download this app on your own phone?



9: What features would you like to see in the in-app UI in future?



10. Have you any other comments or suggestions in relation to the app?

- Check for scam texts too
- Add a bank of pre-existing trusted numbers such as government numbers

Conclusion:

Students found that they were made aware of the potential threat that the warning highlighted. Students' suggestions included making the notification coloured, so it won't be mistaken for a regular notification. Most found that they could clearly understand the urgency of this warning. 70% of students said that they would find this notification useful. 95% of students found the feedback question relating back to the database to be clear enough to understand. Over 50% of students said that they would be very likely to respond to this question. Most people surveyed (68%), said that they would like to have a call log showing suspicious numbers. Another common suggestion amongst students was to have a bank of pre-existing trusted numbers such as government numbers.

App improvements:

Deploying the app to Android 10 (allowing it to operate and be tested on a real device):

Android 10 is the most common version of Android available currently. To deploy this app on a real device the app would need to function on this version. Unfortunately, several issues were encountered as a result of upgrading to Android 10. Google change their APIs, add new libraries and security features on each Android release. As a result of a change like this some features of the app do not work on this version: most prominently the API which fetches an incoming number. The way in which the number is retrieved must change. Using help the Android Documentation, the following changes were made.

Code:

Setting up the 'CallScreeningService' class:

Firstly, go to the AndroidManifest.xml file (*see experimental methods for explanation of this*) and add this block of code to describe the existence of this class to Android. This can be found in the Android Documentation (*see references*)

```
<service
    android:name="com.btys.sb.CallScreeningService"
    android:permission="android.permission.BIND_SCREENING_SERVICE"
    android:exported="false">
    <intent-filter>
        <action android:name="android.telecom.CallScreeningService" />
    </intent-filter>
</service>
<!-- Info about 'CallScreeningService' class -->
```

Next create the 'CallScreeningService' class. Import these classes for use here.

```
package com.btys.sb;
// Package name

import android.os.Build;
import android.telecom.Call;
import androidx.annotation.RequiresApi;
// Classes imported for use in this class
```

Now define the 'CallScreeningService' class. This is what is being used to fetch the incoming number during an incoming phone call. It extends a default Android class whose features are used in getting the number.

```
@RequiresApi(api = Build.VERSION_CODES.Q)
public class CallScreeningService extends android.telecom.CallScreeningService {
```

Add this constant and these variables which will be used throughout this class. As with the 'ServiceReceiver' class it is easier to use these than writing their values where they are needed.

```
public static final String HEX_ASCII_FOR_PLUS_SYMBOL = "%2B";
private static String incomingNumber;
private static boolean incomingCall = false;
// Constant and variables used throughout this class
```

Next, add this method. This is an alternative method to getting an incoming number. It intercepts an incoming call and stores the desired info (phone number) in a URI object as a string in the format "tel:number". This code is what was recommended by the Android Documentation.

```
@Override
public void onScreenCall(Call.Details details) {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
        if (details.getCallDirection() == Call.Details.DIRECTION_INCOMING) {
            android.telecom.CallScreeningService.CallResponse.Builder response =
                new android.telecom.CallScreeningService.CallResponse.Builder();
            response.setDisallowCall(false);
            response.setRejectCall(false);
            response.setSilenceCall(false);
            response.setSkipCallLog(false);
            response.setSkipNotification(false);
            // Method to get incoming number, code sourced from Android Documentation
```

Insert this variable. This will be set to the incoming number. A function is used to split the previously created URI object which stores the calling number as a string in the format 'tel:number'. The colon here is used as a reference of where to split the string and returns a string array object containing the two parts. The second item in the array holds the number and is indexed using the index [1].

```
incomingNumber = details.getHandle().toString().split(regex: ":")[1];
// Variable for incoming number, set by above function
```

As real numbers have a '+' at the start, during testing '%2B' (ASCII hex code for '+') came up at the start of numbers in the app notifications. This code needs to be added check for this value and change it to a human-readable '+' if it is present.

```
if (incomingNumber.startsWith(HEX_ASCII_FOR_PLUS_SYMBOL)) {
    incomingNumber = "+" + incomingNumber.substring(3);
    // Changes ASCII hex '%2B' at start of number to human-readable '+'
}
```

Add this method call, which makes sure that the flow of the incoming call isn't disrupted after the 'onScreenCall' method intercepts it to get the incoming number. This is very important according to the Android Documentation as the call will be dropped if this method is not executed.

```
respondToCall(details, response.build());  
// Makes sure call isn't interrupted, very important according Android Documentation.
```

Add this variable to be set to true. This is another very important line of code, which is used to prevent a strange behaviour from occurring. Previously the feedback notification would appear regardless of whether the call was incoming or outgoing. If this variable is set to true when the 'onScreenCall' method executes during an incoming call, it can be checked later to determine if a call was incoming or not and hence whether to display the feedback notification.

```
        incomingCall = true;  
        // Variable used elsewhere to determine if the call is incoming or outgoing  
    }  
}
```

Insert this method which sets the 'incomingCall' variable back to false for the next instance of this class.

```
public static void setIncomingCall(boolean isIncomingCall) {  
    incomingCall = isIncomingCall;  
    // Resets the 'incomingCall' variable for the next instance of this class  
}
```

Insert this code which return the values of the two variables set earlier: a true or false statement as to whether a call was incoming and the value of an incoming number. These are used in the 'ServiceReceiver' class.

```
public static boolean isIncomingCall() {  
    return incomingCall;  
    // Returns value of 'incomingCall' variable, set earlier  
}  
  
public static final String getIncomingNumber() {  
    return incomingNumber;  
    // Returns value of 'incomingNumber' variable, also set earlier  
}  
}
```

Changes to the 'ServiceReceiver' class:

Firstly, import this class and add these constants for use later.

```
import android.telephony.PhoneStateListener;

private static final int PHONE_RINGING = 1;
private static final int PHONE_ANSWERED = 2;
private static final int PHONE_CALL_ENDED = 0;
```

Next add this method. Its features will replace some parts of the 'onReceive' method: reading the phone state, deciding when to start method to check the contacts list and database for an incoming number and deciding when to display the end of call feedback notification. It makes use of the 'incomingNumber' and 'incomingCall' variables set earlier in the 'CallScreeningService' class.

```
private void registerPhoneStateListener(Context context) {
    /* Replaces parts of the 'onReceive' method, used to get phone
    state and trigger methods to work out which notifications to display */

    TelephonyManager telephony = (TelephonyManager) context.getSystemService(Context.TELEPHONY_SERVICE);
    // Fetches telephony manager from context
    telephony.listen(new PhoneStateListener() {
        // Adds listener at telephony manager to listen for updates to phone call state

        @Override
        public void onCallStateChanged(int phoneState, String incomingNumber) {
            // Executes upon change in call state

            if (phoneState == PHONE_RINGING && previousPhoneState != PHONE_RINGING) {
                // If phone is ringing
                lastIncomingNumber = CallScreeningService.getIncomingNumber();
                // Get incoming number
                handleStartOfCallScamBlockChecking(context, intent: null, lastIncomingNumber);
                // Start method to determine a warning notification to send
            }

            if (phoneState == PHONE_CALL_ENDED && previousPhoneState != PHONE_CALL_ENDED) {
                // If call is ended

                if (previousPhoneState == PHONE_ANSWERED && CallScreeningService.isIncomingCall()) {
                    // If user answered call that was incoming
                    displayEndOfCallNotification(context, lastIncomingNumber);
                    // Display end of call feedback notification
                    CallScreeningService.setIncomingCall(false);
                    // Sets incomingCall variable back to false
                }
            }

            if (phoneState != previousPhoneState) {
                previousPhoneState = phoneState;
                // Saves phone state in static variable for next instance of this class
            }
        }, PhoneStateListener.LISTEN_CALL_STATE);
        // Second parameter of listener: the call state of the phone
    }
}
```

Any code associated with call state in the 'onReceive' method can be removed as checking call state and acting accordingly is now being completed by the method above. However, the code below is still needed in making notifications disappear and the database update, depending on how a user interacts with a notification.

```
@Override
// Annotation that ensures the parameters in the below signature are not breached
public void onReceive(final Context context, final Intent intent) {
    registerPhoneStateListener(context);
    // Implementation of method in the 'BroadcastReceiver' class, which we must provide
    String state = "";
    // Variable to store current state of the phone
    if (intent.getAction().equals(START_OF_CALL_ACTION)) {
        // If user receives notification
        handleStartOfCallUserSelectedDismissOptionSelected(context, intent);
        // Initiate method to remove notification when user presses 'dismiss'
    } else if (intent.getAction().equals(END_OF_CALL_TRUST_ACTION) ||
        intent.getAction().equals(END_OF_CALL_SUSPICIOUS_ACTION)) {
        // If the end of call notification appears and the user interacts with it
        handleEndOfCallTrustOrReportOptionSelectedAction(context, intent);
        // Start method to update database and make notification disappear
    }
    else {
        previousState = state;
        /* Saves current state in this static variable
        for use in the next instance of this class */
    }
}
```

The 'handleEndOfCallAction' method and the associated 'callAnswered' variable can both be removed. Previously displaying the feedback notification was completed over multiple steps, 'handleEndOfCallAction' being one of them. With the new system to check call state this can be done in one step. As a result of this change this method and variable is no longer required.

Adding a call log feature to the database:

Firstly, create a new class called 'IncomingCallLog'. This class contains a constructor to create an object with all the information to be written to the database on a call that took place. This information includes: the date and time, the number that called, and whether the user reported the number as suspicious. It will be called later by a different method (*writeCallLog*).

```
package com.btys.sb;

import android.text.format.DateFormat;
import com.google.firebase.database.IgnoreExtraProperties;
import java.util.Date;

@IgnoreExtraProperties
public class IncomingCallLog {

    public String date;
    public String fromNumber;
    public boolean reportedSuspicious;
    // Declares variables used later

    public IncomingCallLog() {
        // Default constructor required for calls to DataSnapshot.getValue(IncomingCall.class)
    }

    public IncomingCallLog(Date date, String fromNumber, Boolean reportedSuspicious) {
        this.date = String.valueOf(DateFormat.format("dd-M-yy HH:mm:ss", date));
        this.fromNumber = fromNumber;
        this.reportedSuspicious = reportedSuspicious;
        // Creates object for call log with all the desired info, will be added to database
    }
}
```

Add this method to the 'ServiceReceiver' class. This gets a unique ID belonging to user to be written to the call log. This ID is used instead of their number because changes have been made from Android 10 to prevent accessing the phone number of a user's handset mainly for security reasons (*see references*).

```
private String getDeviceId(Context context) {
    return Settings.Secure.getString(context.getContentResolver(),
        Settings.Secure.ANDROID_ID);
    // Method which uses API to get unique constant ID for phone, used in call log
}
```

In the 'handleEndOfCallTrustOrReportOptionSelectedAction' method, add the 'reportSuspicious' (whether the user reported the number as suspicious or trusted) and subscriber (result of 'getDeviceId' method) variables. Call the 'writeCallLog' method.

```
else {
    // If the user reports number
    reportSuspicious(incomingNumber);
    // Initiate system to report suspicious value to the database
    reportSuspicious = true;
    // Used in call log to see if user reported number
}

String subscriber = getDeviceId(context);
// Get user's phone number

writeCallLog(subscriber, incomingNumber, reportSuspicious);
// Start method to write call log info to database for use later
}
```

Add this method to which updates the database call log with all the information previously obtained.

```
public void writeCallLog(String subscriber, String incomingCall,
                        boolean reportedSuspicious) {
    // Method for updating database call log
    IncomingCallLog log = new IncomingCallLog(new Date(), incomingCall,
        reportedSuspicious);
    // Create a new call log object with info on a call that took place
    dbReference.child("incomingCallLogs").child(subscriber).push().setValue(log);
    // Database API call to update its content with the info set above
}
```

Ongoing developments:

This version of the app with the new desirable features has been thoroughly tested on real devices, and any bugs encountered were solved. There are still other features in development though:

A system to check the database call log to see if the user has previously rated a number that is calling them. It will not let the feedback notification appear if they have. This prevents the same person from reviewing a number over and over, which is important as it will prevent scammers from bombarding their own numbers with positive reviews.

An update to the app UI which prompts the user to enter their phone number. The phone number can then be used in conjunction with Firebase phone authentication to register the user with the database. This allows the database to be read from/written to by the user while maintaining an adequate level of security, keeping out any unwanted visitors.